Domain Normalizer — canonical form derivation for noisy identifiers

SPEC-SH-004 v0.1 DRAFT Jeremie Nunez

1 Context

Records arrive from heterogeneous sources using inconsistent spellings for the same domain identifier: mixed case, surrounding whitespace, separator variants (space vs. hyphen vs. underscore), diacritics, stray punctuation, and occasional prefixes or suffixes that are semantically noise. Downstream indexing, deduplication, joins and embeddings all require a single *canonical form* per logical identifier. The domain normalizer is the shared utility that derives this canonical form deterministically from any raw input, so that equality of canonical forms is a reliable proxy for logical equality.

2 Goals

- Produce a deterministic, side-effect-free canonical string from any input string.
- Collapse the common noise axes (case, whitespace, separators, diacritics) without destroying semantic content.
- Preserve reversibility metadata: callers can inspect what was stripped or folded, and recover the original input.
- Be safe on pathological inputs (empty string, only punctuation, very long strings, mixed scripts) without throwing.

3 Non-Goals

Non-Goal

The normalizer does *not* perform fuzzy matching, phonetic encoding (Soundex / Metaphone), translation between languages, or synonym resolution. It does not know any domain ontology: it operates on the string surface. Mapping canonical forms to business entities is the job of a higher-level resolver.

4 Invariants

Invariant

Idempotence: for any input s, $\text{normalize}(\text{normalize}(s)) = \text{normalize}(s)$.

Invariant

Determinism: $\text{normalize}(s)$ depends only on s and on compile-time configuration; no locale, time, or process state influences the output.

Invariant

Non-destruction: if two inputs differ in a character that is neither whitespace, nor a recognised separator, nor a diacritic, nor case, then their canonical forms differ.

5 Interface

```typescript
export interface NormalizeOptions {
  readonly separator?: "-" | "_" | "";    // default: "-"
  readonly stripDiacritics?: boolean;     // default: true
  readonly lowercase?: boolean;           // default: true
  readonly collapseWhitespace?: boolean; // default: true
}

export interface NormalizeResult {
  readonly canonical: string;
  readonly original: string;
  readonly strippedDiacritics: boolean;
  readonly collapsedSeparators: boolean;
}

export function normalizeDomain(
  input: string,
  options?: NormalizeOptions
): NormalizeResult;

export function canonical(
  input: string,
  options?: NormalizeOptions
): string; // shorthand: returns result.canonical
```

6 Scenarios

Scenario: Case, whitespace, diacritics all folded

Given the input `" Côte-Rotie "`
When `normalizeDomain` is called with defaults
Then `canonical` equals `"cote-rotie"`
And `strippedDiacritics` is `true`.

Scenario: Separator variants collapse to one

Given two inputs `"foo bar"`, `"foo_bar"`, `"foo-bar"`
When each is normalised with defaults
Then all three produce `canonical` equal to `"foo-bar"`.

Scenario: Idempotence

Given any input s
When normalization is applied twice
Then the second application returns the same `canonical` string as the first, and the `original` field still reflects the first-stage input.

Scenario: Empty or punctuation-only input

Given inputs `""`, `" "`, `"--"`
When normalization runs
Then `canonical` equals `""`
And no exception is thrown.

Scenario: Non-Latin script preserved

Given input `"Élégant 2024"` with diacritic stripping on
When normalised
Then `canonical` equals `"elegant-2024"`, preserving the digit run as a single token.

7 Acceptance Criteria

Acceptance Criterion AC-1

Default normalization lowercases, strips leading / trailing whitespace, folds diacritics on Latin characters, and replaces any run of whitespace, hyphen, or underscore by a single configured separator (default `"-"`).

Acceptance Criterion AC-2

For every string s, `canonical(canonical(s)) === canonical(s)` (idempotence).

Acceptance Criterion AC-3

Two inputs that differ only in case, whitespace, or separator variant produce identical canonical forms under default options.

Acceptance Criterion AC-4

Two inputs that differ in a non-folded character produce different canonical forms.

Acceptance Criterion AC-5

`normalizeDomain("")` and `normalizeDomain(" ")` both return `canonical === ""` without throwing.

Acceptance Criterion AC-6

When `stripDiacritics` is `false`, diacritics are preserved verbatim in the canonical form.

Acceptance Criterion AC-7

The `original` field on the result always equals the input passed in, regardless of options.

Acceptance Criterion AC-8

On inputs up to 10^5 characters, normalization completes in under 10 ms on the reference CI runner and allocates $O(n)$ memory.

8 Traceability

AC	Test file	Test name
AC-1	`packages/shared/tests/domain-normalizer.spec.ts`	`default options fold case, whitespac`
AC-2	`packages/shared/tests/domain-normalizer.spec.ts`	`normalization is idempotent`
AC-3	`packages/shared/tests/domain-normalizer.spec.ts`	`separator variants collapse to canon`
AC-4	`packages/shared/tests/domain-normalizer.spec.ts`	`non-folded characters remain disting`
AC-5	`packages/shared/tests/domain-normalizer.spec.ts`	`empty and whitespace-only inputs ret`
AC-6	`packages/shared/tests/domain-normalizer.spec.ts`	`stripDiacritics=false preserves acce`
AC-7	`packages/shared/tests/domain-normalizer.spec.ts`	`original field round-trips the input`
AC-8	`packages/shared/tests/domain-normalizer.bench.ts`	`normalizes 100k-character input unde`

9 Open Questions

- Should non-Latin diacritic-bearing scripts (e.g. Cyrillic, Greek) be folded by the same NFD+strip pass, or preserved?
- Is an optional `maxLength` truncation parameter in scope, or left to callers?
- Should the API expose a streaming variant for very large bulk ingestion, or is the synchronous signature sufficient?